DEHIX Live Room

Detailed Product, Feature, Architecture, and Future Roadmap Document

Prepared on: June 10, 2026

Production handoff document for frontend, backend, AI workflows, collaboration room, and roadmap.

Platform Type	AI-powered Web3 hiring and project execution platform
Frontend	React, Vite, TypeScript, Tailwind CSS, Socket.IO client
Backend	Express, MongoDB, Mongoose, Socket.IO, Azure OpenAI
Core Outcome	Convert raw ideas into validated, scoped, staffed, and managed project rooms

Contents

This contents section lists the major areas covered in the project document.

1. Executive Summary
2. Problem We Are Trying To Solve
3. Product Vision
4. Target Users
5. What We Have Built

5.1 Frontend Application

5.2 Landing Page

5.3 Authentication

5.4 Business Dashboard

5.5 Talent Dashboard

5.6 Talent Discovery

5.7 Talent Profile

5.8 Business Idea Validation Flow

5.9 Phase 1 Review and Confirmation

5.10 Business Validation PDF

5.11 Technical Intake Questions

5.12 Business and Development Blueprint

5.13 Talent Recommendations

5.14 Live Room Creation

5.15 Live Room Workspace

5.16 Ticket Management

5.17 Milestone Management

5.18 NDA Workflow

5.19 AI Assistant

5.20 AI-Generated Documents

5.21 Room Activity Timeline

5.22 Realtime Updates

5.23 Firebase Live Chat Support

5.24 Room Join Flow

5.25 Exports and Downloads

6. Backend Architecture

6.1 Backend Route Areas

6.2 Data Models

7. Frontend Architecture
8. Security and Production Readiness Work Completed

8.1 API Security Headers

8.2 Rate Limiting

8.3 Production CORS Allowlist

8.4 JWT Secret Enforcement

8.5 Room Authorization

8.6 Socket.IO Authentication

8.7 Frontend Production Config

9. Environment Variables

10. Current Verification Status

11. Current Limitations

12. Future Roadmap

12.1 Short-Term Roadmap

12.2 Medium-Term Roadmap

12.3 Long-Term Roadmap

13. Recommended Production Launch Checklist

14. Business Value

15. Conclusion

Prepared on: June 10, 2026

1. Executive Summary

DEHIX Live Room is a full-stack, AI-powered Web3 hiring and project execution platform. The product helps a business owner move from a raw startup or product idea to a structured execution room where the project can be validated, scoped, staffed, tracked, documented, and contracted.

The platform combines:

- Business idea validation.
- AI-generated business and technical planning.
- Technical intake and blueprint generation.
- Talent discovery based on simulated Web3 credentials.
- Real-time room collaboration.
- AI-assisted project management.
- Ticket, milestone, NDA, and document workflows.
- Production-focused backend security and room access controls.

The current version is suitable as a strong demo, staging, or production-handoff build. It now includes core frontend and backend flows, MongoDB persistence, Azure OpenAI integration, Firebase-powered optional live chat, Socket.IO room updates, API hardening, and deployment documentation.

2. Problem We Are Trying To Solve

Web3 project hiring is difficult because businesses often start with only a rough idea and do not know how to convert that idea into a clear build plan. Freelancers and technical teams also need structured information before they can estimate cost, timeline, risk, and team requirements.

Typical problems in the current process:

- Business owners do not always know whether an idea is viable before hiring developers.
- A raw idea is not enough for technical execution.
- Hiring is fragmented across chats, spreadsheets, documents, freelancer profiles, and manual interviews.
- Businesses struggle to define the right Web3 roles, skills, reputation level, and budget.
- Freelancers need project context, milestones, tickets, and payment expectations before joining.
- NDAs, statements of work, technical briefs, and project documents are often prepared manually.
- Teams lack a single room where project scope, AI context, team members, tickets, milestones, documents, and communication live together.
- There is little trust signal in many freelancer marketplaces, especially for Web3 skills such as Solidity, ZK proofs, smart contract security, Rust, Node.js, React, DeFi, and blockchain architecture.

DEHIX Live Room solves this by turning the hiring process into a guided AI workflow. The platform validates the business idea, collects technical requirements, generates a blueprint, recommends team structure, supports talent discovery, and creates a dedicated live room for collaboration and execution.

3. Product Vision

The long-term vision is to become a complete AI-powered Web3 project launch and hiring workspace.

The intended user journey is:

1. A business enters a raw project idea.
2. AI validates the idea from a business perspective.
3. The business reviews market, audience, competitor, risk, revenue, SWOT, and score analysis.
4. The business answers technical intake questions.
5. AI generates a development blueprint, MVP scope, roles, budget, milestones, and tickets.

6. The business discovers and invites relevant talent.
7. The project moves into a live room.
8. The team collaborates in real time.
9. AI helps generate NDAs, documents, tickets, milestones, summaries, and execution guidance.
10. Future releases can connect escrow, GitHub, blockchain credentials, and production contract workflows.

4. Target Users

4.1 Business Users

Business users are founders, product owners, companies, or clients who want to build a Web3 product but need help validating, planning, hiring, and managing execution.

They need:

- Idea validation.
- Product and business analysis.
- Technical scoping.
- Budget and timeline estimation.
- Role recommendations.
- Access to qualified talent.
- Project room creation.
- NDA and milestone workflows.
- Structured documentation.
- A simple dashboard to manage rooms and teams.

4.2 Talent Users

Talent users are freelancers, developers, designers, architects, auditors, or specialists with Web3 skills.

They need:

- A talent profile.
- Visibility through credentials and reputation.
- Invitations to suitable rooms.
- Room participation.
- Milestone and ticket visibility.
- NDA signing.
- Project context before accepting work.
- A dashboard for active rooms and earnings visibility.

4.3 Admin or Platform Team

Although a dedicated admin panel is not yet implemented, the platform team will eventually need:

- User management.
- Credential verification.
- Dispute review.
- Room monitoring.
- Payment and escrow oversight.
- AI usage monitoring.
- Audit trails.

- Analytics and reporting.

5. What We Have Built

5.1 Frontend Application

The frontend is a React 19 and Vite application using TypeScript, Tailwind CSS, Radix UI components, TanStack Query, Wouter routing, Socket.IO client, Firebase client SDK, and generated API hooks.

Main frontend routes:

- `/` - Landing page.
- `/login` - Login page with demo account shortcuts.
- `/register` - User registration.
- `/room/create` - Business idea validation and room creation workflow.
- `/room/:id` - Live Room workspace.
- `/room/join` - Join room by room code.
- `/business/dashboard` - Business dashboard.
- `/talent/dashboard` - Talent dashboard.
- `/talent/discovery` - Talent discovery and invitation flow.
- `/talent/profile/:id` - Public talent profile.

5.2 Landing Page

The landing page explains the DEHIX Live Room concept and guides users toward the demo or authentication flow.

It presents:

- Platform positioning.
- Core benefits.
- High-level workflow.
- Feature highlights.
- Demo account information.
- Entry points to login, register, create rooms, and join rooms.

5.3 Authentication

Authentication is implemented with backend-issued JWT tokens.

Built authentication capabilities:

- User registration.
- User login.
- Logout endpoint.
- Current user endpoint.
- Profile update endpoint.
- Role support for `business` and `talent`.
- JWT-based API authentication.
- Frontend auth context with saved token and user data.
- Automatic `/api/auth/me` validation when a saved token exists.

Security note:

- Tokens are currently stored in browser local storage. This is workable for the current build, but future production improvement should consider secure HTTP-only cookies or stronger XSS protection controls.

5.4 Business Dashboard

The business dashboard gives business users a workspace to manage their rooms.

Built capabilities:

- View active rooms.
- View closed or past rooms.
- See room statistics.
- See participant counts.
- See ticket and milestone stats.
- View contracted rooms.
- Track total and released milestone value.
- Search rooms.
- Edit profile name and wallet address.
- Navigate into room details.
- Create new launch room flow.

5.5 Talent Dashboard

The talent dashboard gives talent users a personal work hub.

Built capabilities:

- View invitations.
- Accept or decline room invites.
- View active joined rooms.
- View assigned role information.
- View milestone stats.
- Toggle or display availability.
- Edit profile name and wallet address.
- View credential cards.
- See reputation-related information.

5.6 Talent Discovery

Talent discovery helps businesses search and invite talent based on simulated Web3 credentials.

Built capabilities:

- Search talent by skill domain.
- Filter by minimum reputation.
- Filter by online status.
- View profile summary.
- View credential summary.
- View primary skill.
- Select an active business room.
- Invite talent to a room.
- Link invited talent to a room role.

Supported example skill domains include:

- Solidity.

- React.
- Node.js.
- Rust.
- ZK Proofs.
- Security.
- DeFi.
- Frontend.
- Smart Contracts.

5.7 Talent Profile

Talent profile pages provide a public-facing view of an individual talent user.

Built capabilities:

- View user name, email, wallet, and role.
- View credential list.
- View reputation score.
- View GitHub score, interview score, and completed projects where available.
- Invite talent to a selected room.
- Navigate back to discovery or dashboard flows.

5.8 Business Idea Validation Flow

The create room flow begins with a raw business idea.

Current flow:

1. Business enters an idea in normal language.
2. Frontend calls `POST /api/launch`.
3. Backend sends the idea to Azure OpenAI.
4. AI returns structured business validation JSON.
5. Backend stores the result in MongoDB as a `LaunchSession`.
6. Frontend renders a detailed validation report.

The validation result can include:

- Region used.
- Idea summary.
- Market demand.
- Target audience.
- Competitor analysis.
- Competitive moat.
- Revenue model.
- Unit economics.
- Cost estimation.
- Go-to-market strategy.
- Risks.
- Suggestions.
- Assumptions.

- SWOT analysis.
- Dimensional scores.
- Overall score.
- Final verdict.

The purpose of this phase is to avoid starting development blindly. It helps the business understand whether the idea is strong, weak, risky, unclear, or worth pursuing.

5.9 Phase 1 Review and Confirmation

After business validation, the user can review and refine key Phase 1 fields before moving forward.

Built capabilities include editable confirmation fields for:

- Region.
- Idea summary.
- Target audience.
- Business model.
- Competitors.
- Market demand.
- Go-to-market direction.

This allows the business user to correct or refine AI output before the technical planning phase.

5.10 Business Validation PDF

The backend supports downloading a business validation PDF.

Endpoint:

- `GET /api/launch/:id/business-validation.pdf`

Room-level PDF endpoint:

- `GET /api/rooms/:id/business-validation.pdf`

The PDF contains a formatted version of the business validation analysis. Current PDF generation is implemented on the backend and uses generated validation data from the session or room.

5.11 Technical Intake Questions

After idea validation, the user continues to technical intake.

Built capabilities:

- Mandatory technical questions.
- Optional AI-generated technical questions.
- Saved answers.
- Mandatory-answer validation.
- Smart AI suggestions for answers.
- AI-powered refinement of user responses.

Mandatory questions currently cover:

- First users and day-one goal.
- First launch platform.
- Top must-have features.
- Need for accounts, payments, files, chat, maps, AI, blockchain, or third-party integrations.
- Timeline, budget, compliance, existing tools, or data constraints.

This phase converts business-level input into build-ready context.

5.12 Business and Development Blueprint

After technical intake, the backend can generate a detailed business and development blueprint.

Built blueprint content can include:

- Executive summary.
- Problem definition.
- Product strategy.
- MVP definition.
- Target users.
- Technical architecture.
- Recommended stack.
- System components.
- API modules.
- Database entities.
- Risk analysis.
- Development roadmap.
- Team requirements.
- Cost estimation.
- Growth strategy.
- Business strategy.

Endpoint:

- `POST /api/launch/:id/blueprint`

PDF endpoint:

- `GET /api/launch/:id/business-blueprint.pdf`

Room-level PDF endpoint:

- `GET /api/rooms/:id/business-blueprint.pdf`

5.13 Talent Recommendations

After blueprint generation, the system can recommend talent based on the generated team requirements and available credentials.

Endpoint:

- `POST /api/launch/:id/talent-recommendations`

Built recommendation logic uses:

- Required role.
- Skill domain.
- Reputation score.
- Credential level.
- GitHub score.
- Interview score.
- Projects completed.
- Estimated hourly rate.

- Budget distribution.
- Role fit score.

The recommendation engine is not yet a true vector-search or production ML model, but it provides a structured matching layer based on seeded credential data.

5.14 Live Room Creation

The final part of launch flow creates a Live Room.

Endpoint:

- `POST /api/launch/:id/scope`

The created room includes:

- Room code.
- Business owner.
- Launch session link.
- Title.
- Raw description.
- AI scoped brief.
- Status.
- Meet link.

The AI scoped brief can include:

- Project title.
- Project summary.
- Estimated weeks.
- Complexity.
- Roles.
- Milestones.
- Tickets.
- Technical risks.
- Suggested total budget.
- Business validation.
- Business blueprint.
- Talent recommendations.

5.15 Live Room Workspace

The Live Room is the main project execution area.

Main tabs:

- Brief.
- Tickets.
- Milestones.
- NDA.
- Activity.

Built capabilities:

- View room metadata.
- View room code.

- Copy room code.
- Join or invite participants.
- Manage room status.
- Save room notes.
- Save or update meeting link.
- Generate AI brief.
- View AI scoped brief.
- Match talent.
- Generate milestones.
- Generate tickets.
- Create tickets manually.
- Update ticket status.
- Create milestones manually.
- Submit milestones.
- Approve and release milestones.
- Generate NDA.
- Sign NDA.
- Generate project documents from chat context.
- Download documents.
- Download room export.
- Download document ZIP.
- View activity timeline.
- Use AI assistant in the room.

5.16 Ticket Management

Built ticket capabilities:

- Create tickets.
- View room tickets.
- Update ticket status.
- Track status across backlog, todo, in progress, and done.
- Store description.
- Store assigned role.
- Store milestone number.
- Store estimated hours.
- Emit realtime room ticket update events.

Ticket routes:

- `GET /api/rooms/:id/tickets`
- `POST /api/rooms/:id/tickets`
- `PUT /api/tickets/:id`

5.17 Milestone Management

Built milestone capabilities:

- Create milestones.
- View milestones.
- Store title and description.
- Store USD amount.
- Store due date.
- Track status.
- Submit milestone.
- Approve or release milestone.
- Emit realtime milestone update events.
- Record milestone activity.

Milestone statuses include:

- Pending.
- In progress.
- Submitted.
- Approved.
- Released.

Milestone routes:

- `GET /api/rooms/:id/milestones`
- `POST /api/rooms/:id/milestones`
- `PUT /api/rooms/:id/milestones/:milestoneId/submit`
- `PUT /api/rooms/:id/milestones/:milestoneId/approve`
- `PUT /api/rooms/:id/milestones/:milestoneId/status`

5.18 NDA Workflow

Built NDA capabilities:

- AI-generated NDA content.
- Store NDA against a room.
- View NDA.
- Sign NDA.
- Track signatures.
- Move NDA status from draft to pending signatures to signed.
- When enough parties sign, mark the room as contracted.
- Emit realtime NDA signed events.
- Record NDA activity.

NDA routes:

- `POST /api/ai/generate-nda`
- `GET /api/rooms/:id/nda`
- `POST /api/rooms/:id/nda/sign`

5.19 AI Assistant

The AI assistant is available in the launch flow and Live Room.

Built capabilities:

- Context-aware project chat.
- Saved AI conversation history.
- Launch-session chat context.
- Room-level chat context.
- Previous conversation memory.
- AI responses based on business validation, blueprint, room notes, tickets, milestones, participants, and NDA context.
- Chat summary generation.
- AI-assisted document generation.

AI endpoints include:

- `GET /api/ai/chat-history`
- `POST /api/ai/chat`
- `POST /api/ai/chat-summary`
- `POST /api/ai/generate-document`
- `GET /api/ai/documents/:id/pdf`

5.20 AI-Generated Documents

The platform can generate documents from selected chat messages and room context.

Supported document types:

- Pitch Deck.
- Technical Deck.
- Business Development Strategy.
- Statement of Work.
- Project Brief.
- Idea Validation Report.
- Business Requirement Document.
- Project Requirement Document.
- MVP Scope Document.
- Technical Architecture Document.
- Feature List Document.
- Development Roadmap.

Built capabilities:

- Select messages from chat.
- Choose document type.
- Generate document content using Azure OpenAI.
- Save generated document.
- Preview document in modal.
- Download generated document as PDF.
- Include generated documents in ZIP export.

5.21 Room Activity Timeline

The system records important room events.

Activity types include:

- Room created.
- Participant invited.
- Participant joined.
- Participant removed.
- Brief generated.
- NDA generated.
- NDA signed.
- Milestone created.
- Milestone released.
- Ticket created.
- Notes updated.
- Room contracted.
- Room closed.

The activity feed gives the project room a lightweight audit trail.

5.22 Realtime Updates

Realtime updates are implemented with Socket.IO.

Built realtime events include:

- Room join.
- Participant joined.
- Participant invited.
- Participant removed.
- Ticket updated.
- Milestone updated.
- NDA signed.
- Squad formed.
- Room status changed.
- Meeting link updated.
- Notes updated.
- Talent availability changed.

Recent production hardening added:

- JWT authentication for Socket.IO.
- Room access verification before joining a room channel.
- Protection against spoofed availability events.

5.23 Firebase Live Chat Support

Firebase is configured as an optional live chat layer.

Built capabilities:

- Firebase app initialization when config is present.
- Firestore message writing for room chat.
- Graceful disabled state when Firebase variables are missing.

The AI assistant uses backend persisted `AiChatMessage` storage, while Firebase can support live chat style message sharing.

5.24 Room Join Flow

Talent users can join a room by code.

Built capabilities:

- Enter room code.
- Backend normalizes room code.
- Find matching Live Room.
- Create `RoomParticipant`.
- Return room data.
- Emit participant joined event.

This supports simple demo and collaboration workflows.

5.25 Exports and Downloads

Built export capabilities:

- Business validation PDF.
- Business blueprint PDF.
- Generated document PDF.
- Room Markdown export.
- Documents ZIP export.

The ZIP export can include multiple generated project documents and blueprint-based reports.

6. Backend Architecture

The backend is an Express 5 API written in TypeScript and bundled with esbuild.

Core backend technologies:

- Node.js.
- Express.
- MongoDB.
- Mongoose.
- Socket.IO.
- Azure OpenAI SDK.
- JWT authentication.
- bcryptjs password hashing.
- Pino logging.
- Helmet security headers.
- express-rate-limit.
- JSZip.
- Puppeteer for PDF rendering in richer report flows.

6.1 Backend Route Areas

Main route modules:

- `auth.ts` - register, login, logout, current user, profile update.
- `launch.ts` - idea validation, technical questions, blueprint, talent recommendations, launch-to-room scope.
- `ai.ts` - AI brief, match, NDA generation, milestone suggestions, tickets suggestions, chat, summaries, documents.
- `rooms.ts` - room CRUD-like workflow, participants, status, notes, meet link, exports, documents.

- `talent.ts` - talent search, profiles, credentials, availability, invites, joined rooms.
- `tickets.ts` - room ticket operations.
- `milestones.ts` - milestone operations.
- `nda.ts` - NDA read and signing.
- `health.ts` - health check.

6.2 Data Models

MongoDB collections are represented by Mongoose models.

User

Stores:

- Email.
- Hashed password.
- Name.
- Role.
- Avatar URL.
- Wallet address.
- Online status.
- Last seen.

LaunchSession

Stores:

- User owner.
- Raw idea.
- Project title.
- Business goal.
- Target audience.
- Budget range.
- Timeline.
- Project type.
- Research text.
- Technical answers.
- Business document text.
- Technical document text.
- Phase status.
- PDF generation status and cache information.

LaunchClarification

Stores:

- Session id.
- Question.
- Answer.
- Order index.

LiveRoom

Stores:

- Room code.
- Business owner id.
- Launch session id.
- Title.
- Raw description.
- AI scoped brief.
- Status.
- Meeting link.
- Notes.
- Contracted date.

RoomParticipant

Stores:

- Room id.
- User id.
- Role id.
- Status.
- Joined date.

RoomRole

Stores:

- Room id.
- Role title.
- Skill domain.
- Required level.
- Minimum reputation.
- Filled-by user id.
- Status.

SbtCredential

Stores simulated Web3 credentials:

- User id.
- Skill domain.
- Level.
- Reputation score.
- Status.
- GitHub score.
- Interview score.
- Projects completed.
- On-chain transaction reference.
- Issued date.

Ticket

Stores:

- Room id.
- Title.
- Description.
- Assigned role.
- Milestone number.
- Estimated hours.
- Status.

Milestone

Stores:

- Room id.
- Title.
- Description.
- Amount in USD.
- Due date.
- Status.

NDA

Stores:

- Room id.
- Content.
- Signed-by users.
- Status.

AiChatMessage

Stores:

- Thread id.
- Launch session id.
- Room id.
- User id.
- User name.
- Role.
- Message.
- AI flag.

GeneratedDoc

Stores:

- Room id.
- Document type.
- Title.
- Content.
- Message count.

- Created-by user.

RoomActivity

Stores:

- Room id.
- Activity type.
- Actor id.
- Actor name.
- Metadata.
- Created date.

7. Frontend Architecture

The frontend follows a page-based structure with reusable UI components.

Core frontend technologies:

- React 19.
- TypeScript.
- Vite.
- Tailwind CSS.
- Radix UI.
- Lucide icons.
- TanStack Query.
- Wouter.
- Socket.IO client.
- Firebase client SDK.
- Generated API client from OpenAPI/Orval.

Important frontend files:

- `src/App.tsx` - route setup.
- `src/context/AuthContext.tsx` - auth state.
- `src/pages/CreateRoom.tsx` - launch workflow.
- `src/pages/LiveRoom.tsx` - main room workspace.
- `src/pages/BusinessDashboard.tsx` - business dashboard.
- `src/pages/TalentDashboard.tsx` - talent dashboard.
- `src/pages/TalentDiscovery.tsx` - talent search and invite.
- `src/pages/TalentProfile.tsx` - talent profile.
- `src/pages/Login.tsx` - login and demo accounts.
- `src/pages/Register.tsx` - registration.
- `src/pages/JoinRoom.tsx` - join by code.

8. Security and Production Readiness Work Completed

Recent hardening work was added to make the app safer for production handoff.

8.1 API Security Headers

The backend now uses Helmet to set common HTTP security headers.

Purpose:

- Reduce exposure to common browser-level attacks.
- Improve default HTTP security posture.
- Prepare backend for production hosting.

8.2 Rate Limiting

The backend now uses `express-rate-limit`.

Configured limits:

- General API rate limit through `API_RATE_LIMIT`.
- Authentication route rate limit through `AUTH_RATE_LIMIT`.

Purpose:

- Reduce brute-force login attempts.
- Reduce API abuse.
- Add basic production safety.

8.3 Production CORS Allowlist

The backend now uses an explicit CORS allowlist.

Production behavior:

- `CORS_ORIGINS` or `CLIENT_ORIGIN` must be configured.
- If no production origin is configured, the API refuses to start in production.

Development behavior:

- Localhost origins are allowed.

Purpose:

- Prevent arbitrary websites from using browser-based access to the API.
- Align Express and Socket.IO origin policy.

8.4 JWT Secret Enforcement

The backend now refuses to start in production unless `SESSION_SECRET` is configured and at least 32 characters long.

Purpose:

- Avoid accidental deployment with a weak fallback JWT secret.
- Protect user sessions.

8.5 Room Authorization

A shared room access layer was added.

It supports:

- Checking whether a user owns a room.
- Checking whether a user is a joined or accepted participant.
- Middleware for room access.
- Middleware for room owner-only actions.
- Helper for Socket.IO room access checks.

Protected areas now include:

- Room detail.
- Room participants.
- Activity.

- Invite.
- Contract.
- Assemble.
- Close.
- Notes.
- Meeting link.
- Brief.
- Status.
- Participants.
- Tickets.
- Milestones.
- NDA.
- Documents.
- AI room-context routes.
- Socket.IO room joins.

8.6 Socket.IO Authentication

Socket connections now require JWT authentication.

Room join behavior:

- Client sends token during socket connection.
- Backend verifies token.
- Backend checks room access before joining `room:{roomId}`.
- Unauthorized users are blocked.

Purpose:

- Prevent a user from subscribing to realtime events for rooms they do not belong to.

8.7 Frontend Production Config

Vite configuration was tightened.

Changes:

- API proxy default changed to `http://localhost:5001`.
- Host allowlist is controlled by `VITE_ALLOWED_HOSTS`.
- Removed wide-open `allowedHosts: true`.

9. Environment Variables

Important backend variables:

- `NODE_ENV`
- `PORT`
- `MONGODB_URI`
- `SESSION_SECRET`
- `CORS_ORIGINS`
- `CLIENT_ORIGIN`
- `API_RATE_LIMIT`
- `AUTH_RATE_LIMIT`

- JSON_BODY_LIMIT
- TRUST_PROXY
- AZURE_OPENAI_ENDPOINT
- AZURE_OPENAI_API_KEY
- AZURE_OPENAI_API_VERSION
- AZURE_OPENAI_DEPLOYMENT
- AZURE_OPENAI_IMAGE_DEPLOYMENT
- AZURE_OPENAI_AUDIO_DEPLOYMENT
- AZURE_OPENAI_TRANSCRIPTION_DEPLOYMENT

Important frontend variables:

- VITE_API_URL
- VITE_ALLOWED_HOSTS
- FIREBASE_API_KEY
- FIREBASE_APP_ID
- FIREBASE_PROJECT_ID
- BASE_PATH

10. Current Verification Status

The following checks have passed after recent hardening:

```
npm run typecheck
npm run build
npm audit --omit=dev
```

Production dependency audit result:

- `0 vulnerabilities` with `npm audit --omit=dev`.

Known non-blocking build warnings:

- Frontend bundle chunk is larger than 500 kB.
- Sourcemap warnings exist for some UI components.

Known dev-only audit note:

- Plain `npm audit` reports moderate dev dependency issues from an older nested esbuild path through `drizzle-kit`.
- Production audit is clean when dev dependencies are omitted.

11. Current Limitations

The current system is strong for demo and handoff but still has limitations before a full public production launch.

11.1 Testing

There are no formal automated test suites yet.

Needed tests:

- Auth route tests.
- Room authorization tests.
- Socket authorization tests.
- Launch workflow tests.
- AI response parsing tests.
- Ticket and milestone access tests.
- NDA signing tests.

- Frontend smoke tests.

11.2 CI/CD

There is no CI/CD workflow in the repo yet.

Recommended CI checks:

- `npm ci`
- `npm run typecheck`
- `npm run build`
- `npm audit --omit=dev`
- Unit tests when added.
- Integration tests when added.

11.3 AI Output Validation

AI responses are parsed and used, but deeper schema validation should be improved.

Recommended improvements:

- Zod schemas for all AI outputs.
- JSON repair retry for invalid model output.
- Prompt versioning.
- Saved raw AI request and response metadata.
- Token usage tracking.
- Lower temperature for more consistent planning output.

11.4 PDF and Document Formatting

PDF generation works, but professional document output can be improved.

Recommended improvements:

- Consistent branded templates.
- Better page breaks.
- Tables.
- Charts.
- Cover pages.
- Better typography.
- More robust PDF rendering pipeline.

11.5 Frontend Performance

The frontend bundle is currently large.

Recommended improvements:

- Route-level code splitting.
- Dynamic imports for heavy pages.
- Lazy load document/PDF-heavy features.
- Bundle analysis.
- Manual chunk configuration.

11.6 Payment and Escrow

Milestones simulate escrow-style tracking but do not yet connect to real payments or blockchain escrow.

Needed for production payment flow:

- Payment provider integration.
- Wallet connection.
- Smart contract escrow.
- Release conditions.
- Dispute handling.
- Transaction history.

11.7 Real Credential Verification

Credentials are currently simulated.

Needed future work:

- Real SBT or verifiable credential issuance.
- Credential admin review.
- On-chain transaction verification.
- GitHub integration.
- Portfolio verification.
- Skill assessment workflow.

12. Future Roadmap

12.1 Short-Term Roadmap

12.1.1 Testing and Quality

Add:

- Backend unit tests.
- API integration tests.
- Frontend smoke tests.
- Room authorization tests.
- Socket authentication tests.
- AI JSON parsing tests.

Goal:

- Make the codebase safer for production changes and team handoff.

12.1.2 CI/CD Pipeline

Add GitHub Actions or equivalent CI.

Pipeline should run:

- Install dependencies.
- Typecheck.
- Build API.
- Build frontend.
- Run production audit.
- Run tests.

Goal:

- Ensure every pull request is automatically verified.

12.1.3 Better AI Validation

Add strict validation for AI outputs.

Tasks:

- Define Zod schemas for validation, blueprint, recommendations, tickets, milestones, and documents.
- Add repair prompt fallback.
- Save prompt version.
- Store raw output and cleaned output.
- Calculate scores server-side where possible.

Goal:

- Make AI output consistent and production-safe.

12.1.4 Frontend Bundle Optimization

Add route-level lazy loading.

Priority candidates:

- Create Room page.
- Live Room page.
- Talent Discovery.
- Document modal and PDF-related features.

Goal:

- Improve first load performance.

12.2 Medium-Term Roadmap

12.2.1 Real Escrow and Payment System

Build real milestone payment functionality.

Options:

- Stripe escrow-like payment flow.
- Crypto wallet escrow.
- Smart contract escrow on Polygon, Arbitrum, Base, or another network.

Features:

- Fund milestone.
- Submit work.
- Approve work.
- Release funds.
- Handle disputes.
- Show transaction history.

12.2.2 GitHub Integration

Connect tickets to GitHub issues.

Features:

- Create GitHub repository or select existing repository.
- Sync AI-generated tickets to GitHub issues.
- Sync issue status back to room tickets.
- Link PRs to milestones.

- Show engineering progress in room.

12.2.3 Advanced Talent Matching

Improve matching beyond basic credential scoring.

Future matching signals:

- Skills.
- Reputation.
- Past project success.
- GitHub contribution history.
- Availability.
- Time zone.
- Rate expectations.
- Interview score.
- Domain specialization.
- Team fit.

12.2.4 Admin Console

Build an internal admin dashboard.

Capabilities:

- Manage users.
- Review credentials.
- Monitor rooms.
- View AI usage.
- Handle reported issues.
- Resolve disputes.
- Manage demo data.
- View platform analytics.

12.2.5 Professional Document Suite

Expand generated documents.

Add:

- Formal PRD.
- Technical specification.
- Architecture decision record.
- Statement of work.
- Proposal.
- Contract.
- Invoice.
- Milestone acceptance report.
- Investor-ready pitch deck export.

12.3 Long-Term Roadmap

12.3.1 On-Chain Credential Layer

Move from simulated credentials to real verifiable credentials.

Possibilities:

- Soulbound tokens.
- Verifiable credentials.
- Wallet-based profile.
- On-chain project completion history.
- Reputation calculation from completed milestones.

12.3.2 Smart Contract Escrow

Implement production-grade smart contracts.

Features:

- Room-specific escrow contract.
- Milestone-based release.
- Multi-signature approvals.
- Dispute pause.
- Refund logic.
- Event indexing.

12.3.3 AI Project Manager

Turn the AI assistant into an always-on project manager.

Future capabilities:

- Detect blocked tickets.
- Suggest next sprint.
- Generate standup summaries.
- Flag delayed milestones.
- Draft client updates.
- Recommend scope cuts.
- Estimate budget changes.
- Compare planned vs actual progress.

12.3.4 Marketplace Expansion

Expand beyond Web3.

Possible verticals:

- AI product development.
- SaaS development.
- Mobile app development.
- Data engineering.
- Cybersecurity.
- Design and product strategy.

13. Recommended Production Launch Checklist

Before public production launch:

1. Configure production environment variables.
2. Use production MongoDB database.
3. Remove demo seed data.

4. Configure strong `SESSION_SECRET`.
5. Configure `CORS_ORIGINS`.
6. Configure Azure OpenAI deployment.
7. Configure Firebase only if live chat is needed.
8. Run `npm ci`.
9. Run `npm run build`.
10. Run `npm audit --omit=dev`.
11. Add CI/CD.
12. Add automated tests.
13. Add monitoring and error tracking.
14. Add database backup policy.
15. Add log retention policy.
16. Add production domain and HTTPS.
17. Add real email/password recovery if needed.
18. Add admin controls before real users are onboarded at scale.

14. Business Value

DEHIX Live Room creates value by reducing the gap between idea and execution.

For businesses:

- Faster project scoping.
- Lower hiring confusion.
- Better planning before spending money.
- Clearer milestone and budget structure.
- Access to relevant Web3 talent.
- AI-generated documents and summaries.

For talent:

- Better project clarity.
- More relevant invitations.
- Credential-based visibility.
- Clear milestone expectations.
- Room-based collaboration.

For the platform:

- Differentiated AI-first Web3 hiring workflow.
- Stronger trust layer through credentials.
- Better retention through project rooms.
- Future revenue opportunities through escrow, premium AI planning, talent placement, and SaaS subscriptions.

15. Conclusion

DEHIX Live Room is no longer just a simple marketplace demo. It is a structured AI-powered project launch and collaboration platform.

The current build includes:

- Business idea validation.
- Technical intake.

- AI blueprint generation.
- Talent discovery.
- Credential-based matching.
- Live room collaboration.
- Tickets.
- Milestones.
- NDA workflow.
- AI assistant.
- AI-generated documents.
- PDF and ZIP exports.
- Room activity tracking.
- Authentication.
- Production-focused backend hardening.

The most important next steps are testing, CI/CD, stronger AI schema validation, performance optimization, and real production deployment setup. After those are complete, the platform can move from production handoff to production launch readiness.